

Metoda wektorów nośnych

Support Vector Machines (SVM)

Adam Zagdański

Wydział Matematyki PWr

Wrocław
22 V 2026

Wprowadzenie
Przypadek klas liniowo separowalnych
Przypadek klas nieseparowalnych liniowo
Konstrukcja nieliniowych powierzchni separujących
Przypadek K-klas
Wady i zalety
Dodatek: SVM w pakiecie R

Plan prezentacji

- 1 Wprowadzenie
- 2 Przypadek klas liniowo separowalnych
- 3 Przypadek klas nieseparowalnych liniowo
- 4 Konstrukcja nieliniowych powierzchni separujących
- 5 Przypadek K-klas
- 6 Wady i zalety
- 7 Dodatek: SVM w pakiecie R

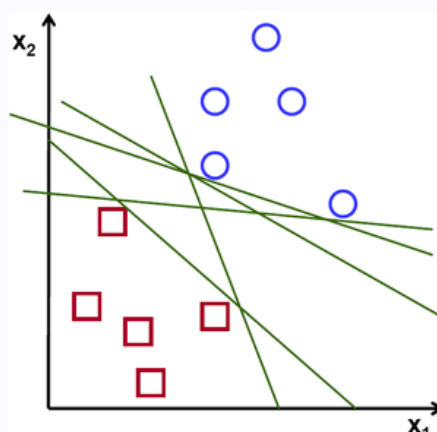
Wprowadzenie

- Metoda wektorów nośnych została zaproponowana przez V.Vapnika (1995).
- Inne nazwy: maszyna wektorów nośnych, maszyna wektorów podpierających, ang. Support Vector Machines (SVM).
- **Idea metody:** znalezienie optymalnej hiperpłaszczyzny rozdzielającej punkty należące do dwóch klas (zagadnienie klasyfikacji binarnej).
- W odróżnieniu od innych liniowych metod klasyfikacji (np. LDA lub LR), SVM stanowi bezpośrednie podejście do konstrukcji hiperpłaszczyzny dyskryminacyjnej
- W praktyce, SVM sprowadza się do rozwiązywania zadania optymalizacji kwadratowej funkcji celu z liniowymi ograniczeniami.

Przypadek klas liniowo separowalnych

Konstrukcja optymalnej hiperpłaszczyzny dyskryminacyjnej

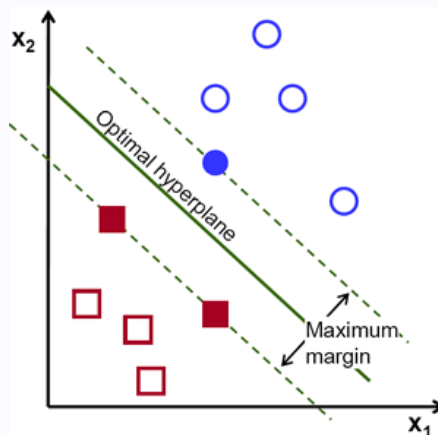
- Na początek rozważmy przypadek klas liniowo separowalnych, tzn. takich które można idealnie rozdzielić hiperpłaszczyzną dyskryminacyjną.
- **Problem:** Jak znaleźć optymalną hiperpłaszczyznę dyskryminacyjną?



Przypadek klas liniowo separowalnych

Konstrukcja optymalnej hiperpłaszczyzny dyskryminacyjnej

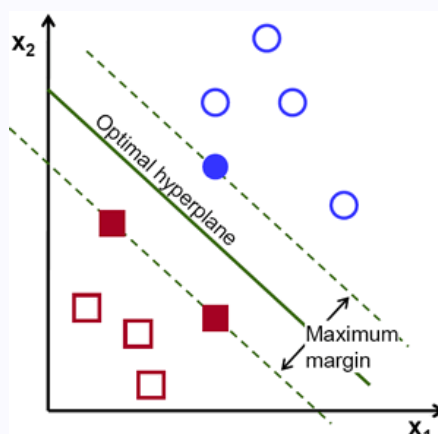
- **Idea:** Szukamy najszerszego „marginesu” ograniczonego dwiema równoległymi hiperpłaszczyznami (wewnątrz marginesu nie znajduje się ani jeden element próby uczącej).
- W środku marginesu umieszcza się hiperpłaszczyznę dyskryminacyjną.



Przypadek klas liniowo separowalnych

Wektory nośne (ang. support vectors)

- Optymalne hiperpłaszczyzny (wyznaczające margines) muszą przechodzić przez pewne elementy próby uczącej (oznaczone jako ■ i ●).
- Są to tzw. wektory podpierające lub wektory nośne (ang. support vectors).



Przypadek klas liniowo separowalnych

Konstrukcja optymalnej hiperpłaszczyzny dyskryminacyjnej

- Próba ucząca: $(x_1, y_1), \dots, (x_n, y_n)$, gdzie $x_i \in \mathbb{R}^p$
- Kodowanie klas: $y_i \in \{-1, 1\}$
- Hiperpłaszczyzna rozdzielająca klasy

$$H : w^T x + b = w \circ x + b = 0,$$

(gdzie ' $\circ$ ' oznacza iloczyn skalarny).

- Dla każdej obserwacji x_i :

$$\begin{cases} x_i \circ w + b \geq +1, & \text{gd}y \ y_i = +1, \\ x_i \circ w + b \leq -1, & \text{gd}y \ y_i = -1, \end{cases}$$

- Równoważnie: $y_i(x_i \circ w + b) - 1 \geq 0$ dla $i = 1, \dots, n$.

Przypadek klas liniowo separowalnych

Konstrukcja optymalnej hiperpłaszczyzny dyskryminacyjnej

- Hiperpłaszczyzny wyznaczające margines:

$$H_1 : x \circ w + b = +1$$

$$H_2 : x \circ w + b = -1$$

- Wyznaczamy odległości

$$d(H_1, 0) = \frac{|1 - b|}{\|w\|}$$

$$d(H_2, 0) = \frac{|-1 - b|}{\|w\|}$$

Odległość między hiperpłaszczyznami:

$$d(H_1, H_2) = \frac{2}{\|w\|} =^{\text{ozn.}} 2M \quad (\text{„margines”})$$

Przypadek klas liniowo separowalnych

Konstrukcja optymalnej hiperpłaszczyzny dyskryminacyjnej

- Warunek wyznaczający optymalną hiperpłaszczyznę dyskryminacyjną H :
maksymalizujemy $\frac{2}{\|w\|}$ (równoważnie: minimalizujemy $\frac{\|w\|^2}{2}$)
przy ograniczeniach: $y_i(x_i \circ w + b) - 1 \geq 0, \quad i = 1, 2, \dots, n.$
- W praktyce, do rozwiązania powyższego zadania optymalizacyjnego stosujemy programowanie kwadratowe (ang. Quadratic Programming (QP))
($\Rightarrow$ szczegóły np. Koronacki i Ćwik, rozdz.6.2).

Przypadek klas liniowo separowalnych

Reguła decyzyjna

- Funkcja decyzyjna (klasyfikator):

$$\hat{G}(x) = \text{sgn}[x^T \hat{w} + \hat{b}].$$

- Można pokazać, że:

$$\hat{G}(x) = \text{sgn} \left[\sum_{\text{wektory wspierające}} y_i \alpha_i^0 (x_i \circ x) + b^0 \right],$$

gdzie α_i^0 – optymalne współczynniki Lagrange'a.

- **Uwaga:** na postać hiperpłaszczyzny dyskryminacyjnej wpływ mają tylko wektory nośne.

Przypadek klas nieseparowalnych liniowo

Elements of Statistical Learning (2nd Ed.) ©Hastie, Tibshirani & Friedman 2009 Chap 12

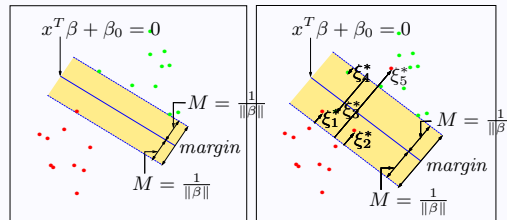


FIGURE 12.1. Support vector classifiers. The left panel shows the separable case. The decision boundary is the solid line, while broken lines bound the shaded maximal margin of width $2M = 2/\|\beta\|$. The right panel shows the nonseparable (overlap) case. The points labeled ξ_j^* are on the wrong side of their margin by an amount $\xi_j^* = M\xi_j$; points on the correct side have $\xi_j^* = 0$. The margin is maximized subject to a total budget $\sum \xi_i \leq \text{constant}$. Hence $\sum \xi_j^*$ is the total distance of points on the wrong side of their margin.

Przypadek klas nieseparowalnych liniowo

- Dopuszczamy aby niektóre punkty znajdowały się po niewłaściwej stronie marginesu.
- Dla tych punktów wprowadzamy współczynniki $\xi_i \geq 0$, $i = 1, \dots, n$.
- Wówczas

$$\begin{cases} x_i \circ w + b \geq 1 - \xi_i, & \text{gdy } y_i = +1, \\ x_i \circ w + b \leq -(1 - \xi_i), & \text{gdy } y_i = -1, \end{cases}$$

- Równoważnie: $y_i(x_i \circ w + b) \geq 1 - \xi_i$, dla $i = 1, \dots, n$.

Przypadek klas nieseparowalnych liniowo

- Warunek wyznaczający hiperpłaszczyznę dyskryminacyjną H : minimalizujemy $\frac{\|w\|^2}{2} + C \cdot \sum_{i=1}^n \xi_i$, pod warunkiem $y_i(x_i \circ w) + b \geq 1 - \xi_i$.
- C – współczynnik kary (parametr kosztu) za niespełnienie „idealnych” ograniczeń.
- **Uwaga 1:** dla punktów po właściwej stronie: $\xi_i = 0$.
- **Uwaga 2:** Całkowita odległość punktów znajdujących się po złej stronie: $\sum_{i=1}^n \xi_i^* = \sum_{i=1}^n M \cdot \xi_i = \sum_{i=1}^n \frac{1}{\|w\|} \cdot \xi_i$.

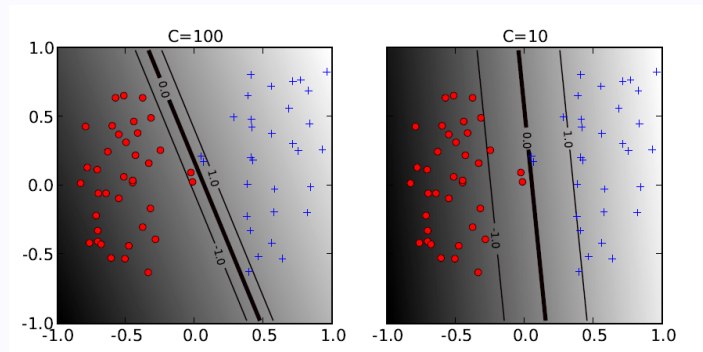
Przypadek klas nieseparowalnych liniowo

Wybór stałej C

- $C = \infty$ – przypadek separowalny liniowo.
- Aby wybrać optymalną wartość parametru kosztu C możemy wykorzystać ocenę prawdopodobieństwa błędnej klasyfikacji np. na bazie schematu cross-validation.
- **Pytanie:** Jak wybór parametru kosztu C wpływa na postać reguły decyzyjnej?

Przypadek klas nieseparowalnych liniowo

Wpływ parametru kosztu C na postać reguły decyzyjnej



- Duża wartość parametru kosztu C wiąże się z dużą karą przypisaną błędnie klasyfikowanym przypadkom.
- Mniejsza wartość C prowadzi do szerszego marginesu (zwiększa się liczba wektorów nośnych).

Przypadek klas nieseparowalnych liniowo

Przykład: Wybór stałej C

Elements of Statistical Learning (2nd Ed.) ©Hastie, Tibshirani & Friedman 2009 Chap 12

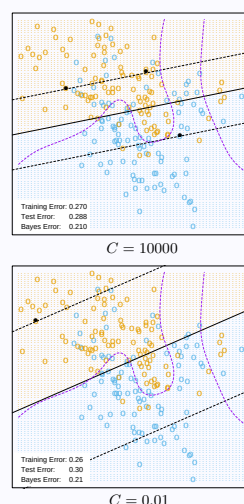


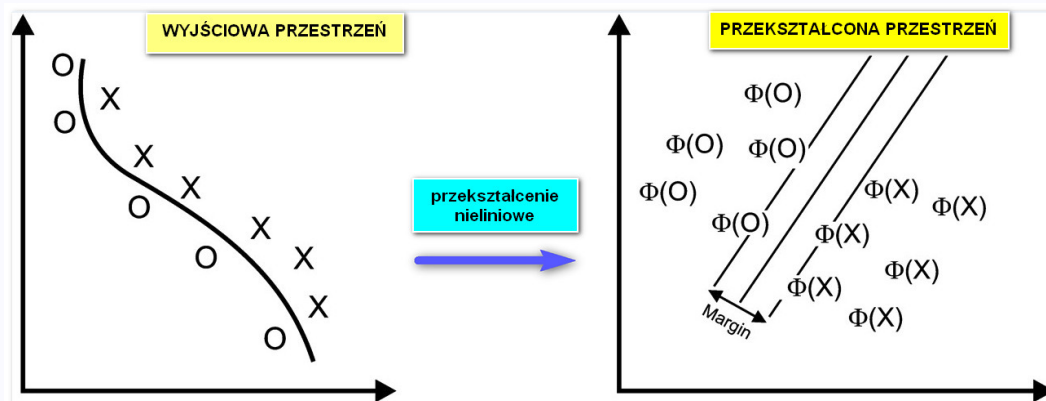
FIGURE 12.2. The linear support vector boundary for the mixture data example with two overlapping classes, for two different values of C . The broken lines

Porównanie:

Dla $C = 10000$
 (górny wykres) 62% obserwacji to wektory nośne, podczas gdy dla $C = 0,01$ (dolny wykres) aż 85%.

Konstrukcja nieliniowych powierzchni separujących

- **Idea:** Transformujemy obserwacje do bogatszej przestrzeni (wyższego wymiaru), w której funkcja separująca klasy będzie liniowa.



Konstrukcja nieliniowych powierzchni separujących

- Transformujemy wyjściowe obserwacje do bogatszej przestrzeni:

$$\Phi(x_i) = (\Phi_1(x_i), \Phi_2(x_i), \dots, \Phi_m(x_i))$$

- Nieliniowa funkcja decyzyjna

$$\hat{f}(x) = [\Phi(x)]^T \cdot \hat{w} + \hat{b}.$$

- Klasyfikator (reguła decyzyjna)

$$\hat{G}(x) = \text{sgn}(\hat{f}(x))$$

- Równoważnie:

$$\hat{G}(x) = \text{sgn}\left(\sum_{\text{wektory podpierające}} y_i \cdot \alpha_i^0 (\Phi(x_i) \circ \Phi(x)) + b^0\right)$$

- **Problem:** Jak obliczyć wartość iloczynu skalarnego?

Konstrukcja nieliniowych powierzchni separujących

Kernel trick

- Do wyznaczenia iloczynu skalarnego wykorzystujemy odpowiednią funkcję jądrową K , która pozwala na operowanie na wielowymiarowej przestrzeni cech bez konieczności obliczania poszczególnych współrzędnych (tzw. "kernel trick")

$$K(x, x') = \Phi(x) \circ \Phi(x').$$

- Taka operacja jest często obliczeniowo dużo mniej złożona niż jawne obliczanie współrzędnych.
- Klasyfikator przyjmuje postać:

$$\hat{G}(x) = \text{sgn}\left(\sum_{\text{wektory podpierające}} y_i \cdot \alpha_i^0 K(x_i, x) + b^0\right)$$

Konstrukcja nieliniowych powierzchni separujących

Najpopularniejsze funkcje jądrowe

- Jądro liniowe

$$K(x, x') = x \circ x'$$

- Jądro wielomianowe stopnia d

$$K(x, x') = (1 + x \circ x')^d,$$

- Jądro gaussowskie (jądro radialne, RBF)

$$K(x, x') = \exp(-\gamma \|x - x'\|^2),$$

- Jądro sigmoidalne

$$K(x, x') = \tanh(\kappa_1 \cdot x \circ x' + \kappa_2)$$

Konstrukcja nieliniowych powierzchni separujących

Wybór funkcji jądrowej a postać granic decyzyjnych

Elements of Statistical Learning (2nd Ed.) ©Hastie, Tibshirani & Friedman 2009 Chap 12

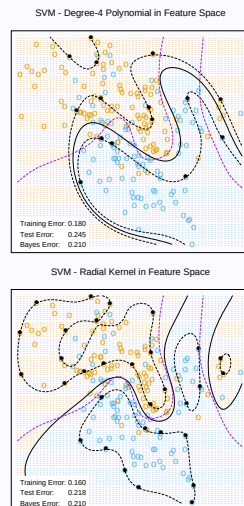


FIGURE 12.3. Two nonlinear SVMs for the mixture data. The upper plot uses a 4th degree polynomial

Wniosek: Jądro radialne (gaussowskie) działa najlepiej i prowadzi do granic decyzyjnych zbliżonych do optymalnych, które otrzymujemy dla klasyfikatora Bayesa (przerywana fioletowa linia).

(C) 2026, A.Zagdański, Metoda wektorów nośnych

21/38

Konstrukcja nieliniowych powierzchni separujących

Najpopularniejsze funkcje jądrowe

Przykład: rozważmy 2-wymiarową przestrzeń cech (zmienne X_1 i X_2). Niech K – jądro wielomianowe stopnia 2.

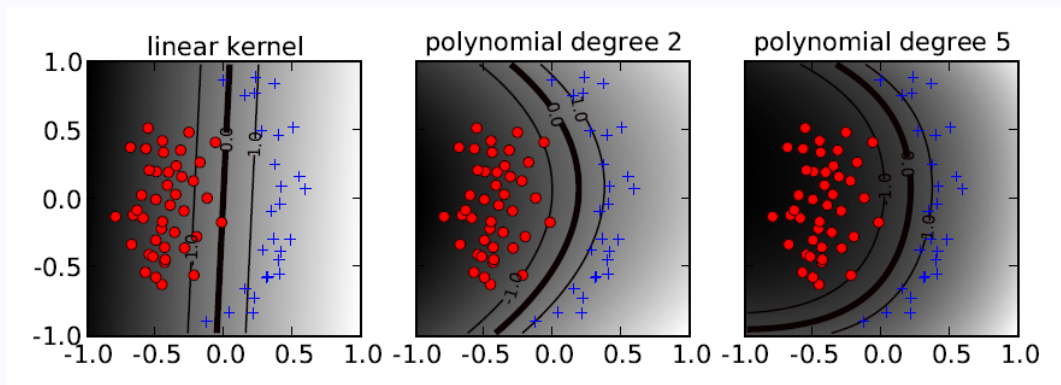
- Niech $x = (x_1, x_2)$ i $x' = (x'_1, x'_2)$.
- Wówczas: $K(x, x') = (1 + x \circ x')^2 = (1 + x_1x'_1 + x_2x'_2)^2 = (1 + x_1x'_1)^2 + 2(1 + x_1x'_1)(x_2x'_2) + (x_2x'_2)^2 = 1 + 2x_1x'_1 + 2x_2x'_2 + (x_1x'_1)^2 + (x_2x'_2)^2 + 2x_1x'_1x_2x'_2$.
- W tym przypadku $m = 6$.
- Jeżeli określimy: $\Phi_1(x) = 1$, $\Phi_2(x) = \sqrt{2}x_1$, $\Phi_3(x) = \sqrt{2}x_2$, $\Phi_4(x) = x_1^2$, $\Phi_5(x) = x_2^2$ oraz $\Phi_6(x) = \sqrt{2}x_1x_2$, wówczas

$$K(x, x') = \Phi(x) \circ \Phi(x').$$

Konstrukcja nieliniowych powierzchni separujących

Wybór jądra a postać reguły decyzyjnej

- Parametry funkcji jądrowej mają istotny wpływ na postać granicy decyzyjnej.
- Stopień jądra wielomianowego d i parametr γ dla jądra gaussowskiego kontrolują elastyczność (złożoność) klasyfikatora.

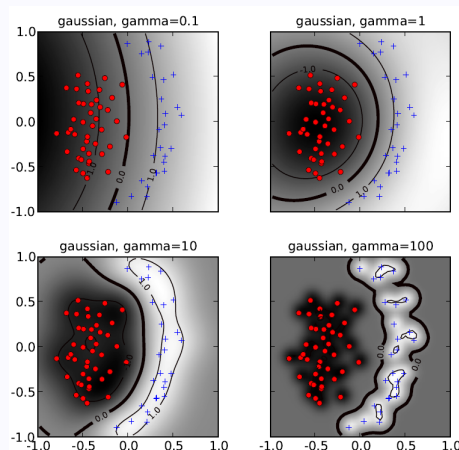


(źródło: A.Ben-Hur and J.Weston, *User's Guide to Support Vector Machines*)

Konstrukcja nieliniowych powierzchni separujących

Wybór jądra a postać reguły decyzyjnej

- Dla ustalonego parametru kosztu C , im większa wartość parametru γ tym większa elastyczność granic decyzyjnych.

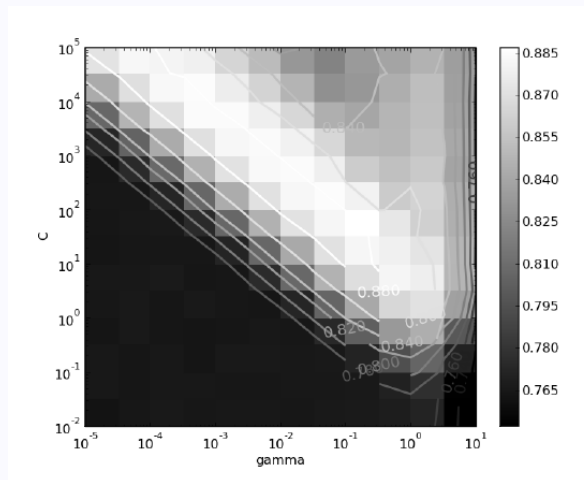


(źródło: A.Ben-Hur and J.Weston, *User's Guide to Support Vector Machines*)

Konstrukcja nieliniowych powierzchni separujących

Wybór jądra a postać reguły decyzyjnej

- Optimalizując jednocześnie C i γ optymalny klasyfikator możemy otrzymać dla różnych kombinacji tych parametrów.

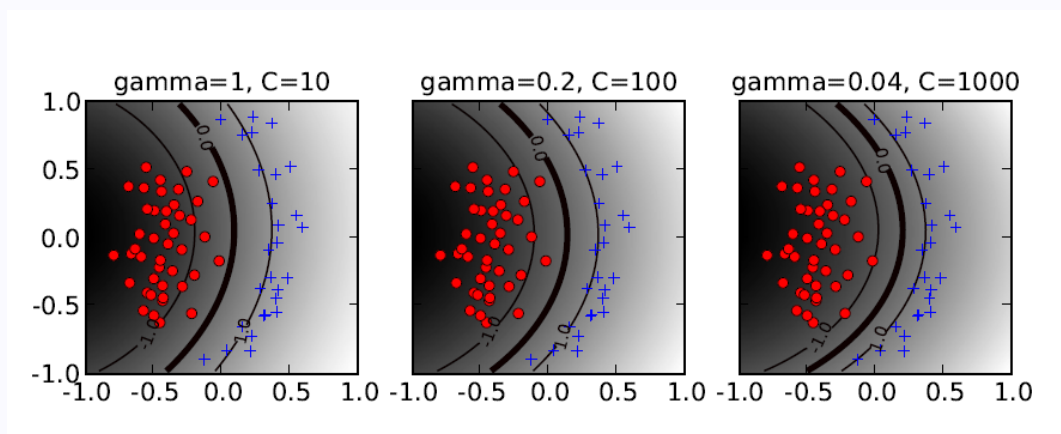


(źródło: A.Ben-Hur and J.Weston, *User's Guide to Support Vector Machines*)

Konstrukcja nieliniowych powierzchni separujących

Wybór jądra a postać reguły decyzyjnej

- Jeżeli zwiększymy wartość parametru kosztu C , aby odpowiednio modelować złożoną granicę decyzyjną powinniśmy zmniejszyć parametr γ .



(źródło: A.Ben-Hur and J.Weston, *User's Guide to Support Vector Machines*)

Przypadek K-klas

Strategia „one-vs-all”

- Algorytm SVM został opracowany dla przypadku klasyfikacji binarnej ($K = 2$).
- Możliwe uogólnienie: stosujemy jedną z metod dekompozycji problemu K klasowego na K zagadnień binarnych.
- Schemat „one-vs-all”: konstruujemy K reguł decyzyjnych, stosując za każdym razem algorytm SVM dla klasyfikacji obiektów z klasy k vs. pozostałe klasy, gdzie $k \in \{1, 2, \dots, K\}$.

$$\hat{G}_k(x) = \text{sgn}(\hat{f}_k(x)), \quad k = 1, 2, \dots, K.$$

- Ostateczna reguła decyzyjna

$$\hat{G}(x) = \text{argmax}\{\hat{f}_1(x), \dots, \hat{f}_K(x)\}.$$

Przypadek K-klas

Inne strategie

- Metoda „one-vs-one” – konstruujemy $\frac{K(K-1)}{2}$ klasyfikatorów dla wszystkich możliwych par klas.
- Oprócz metod opartych na dekompozycji problemu wieloklasowego do wielu zagadnień klasyfikacji binarnej, pojawiały się również próby uogólnienia zagadnienia optymalizacyjnego określonego w SVM na przypadek k -klas (gdzie $k > 2$).
- Przykładem takiego podejścia jest metoda „spoc-svc” (Crammer i Singer (2000)).

Najważniejsze zalety metody SVM

- Dobre własności uogólniające.
- Metoda jest efektywna nawet dla bardzo dużej liczby cech (w szczególności przypadek $p \gg n$).
- Potwierdzone bardzo dobre wyniki w różnych obszarach zastosowań.
- Solidne podstawy teoretyczne oparte na klasycznych metodach optymalizacji.
- Metoda uniwersalna i elastyczna – łatwe tworzenie skomplikowanych modeli nieliniowych.

Najważniejsze wady metody SVM

- Problem z wyborem optymalnych parametrów (m.in. optymalnej funkcji jądrowej i jej parametrów).
- Nieodpowiedni wybór jądra może prowadzić do istotnego zwiększenia błędu klasyfikacji.
- Słaba skalowalność – duża złożoność obliczeniowa i duża zajętość pamięci.
- Problem z interpretacją modelu.
- Brak obsługi zmiennych jakościowych.

Dodatek: SVM w pakiecie R

- funkcja `svm()` w pakiecie `e1071`,
- funkcja `svmlight()` w pakiecie `klaR`,
- funkcja `ksvm()` w pakiecie `kernlab`,
- funkcja `svmpath()` w pakiecie `svmpath`.
- Uwaga: Większość dostępnego oprogramowania umożliwiającego wykorzystanie metody SVM, to implementacje w języku C lub C++ (np. często nagradzana biblioteka `libsvm` (<https://www.csie.ntu.edu.tw/~cjlin/libsvm/>))
- Więcej szczegółów: A.Karatzoglou and D.Meyer, *Support Vector Machines in R*, Journal of Statistical Software 2006, Volume 15, Issue 9.

Dodatek: SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214$, $p = 9$, $K = 6$)

```
library(e1071)
library(mlbench)

#####
# Dane

data(Glass)
dane <- Glass
n <- nrow(dane)
learn.ind <- sample(1:n, 2/3*n)
training.set <- dane[learn.ind,]
test.set <- dane[-learn.ind,]
```


SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Budujemy model SVM (K - jądro liniowe, parametr kosztu: C = 1)  
  
> svm.linear <- svm(Type~., data=training.set, kernel="linear")  
> summary(svm.linear)  
#  
# Call: svm(formula = Type ~ ., data = training.set, kernel = "linear")  
#  
# Parameters:  
#   SVM-Type: C-classification  
# SVM-Kernel: linear  
#     cost: 1  
#   gamma: 0.1111111  
#  
# Number of Support Vectors: 110 ( 40 41 7 12 5 5 )  
#  
# Number of Classes: 6  
# Levels: 1 2 3 5 6 7
```

SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Wyznaczamy prognozowane etykiety dla zbioru testowego  
> real.labels <- test.set$Type  
> n.test <- length(real.labels)  
> pred.svm.lin <- predict(svm.linear, newdata=test.set)  
  
# macierz pomyłek (confusion matrix)  
> table(pred.svm.lin, real.labels)  
real.labels  
pred.svm.lin 1 2 3 5 6 7  
1 13 4 3 0 0 0  
2 14 19 2 5 0 2  
3 0 0 0 0 0 0  
5 0 0 0 3 0 0  
6 0 0 0 0 2 0  
7 0 0 0 0 1 4  
# dokładność klasyfikacji  
> (acc.svm.lin <- sum(diag(table(pred.svm.lin, real.labels)))/n.test)  
# 0.5694444
```

SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Wpływ parametru kosztu C na postać modelu  
  
> svm.linear.C1 <- svm(Type~., data=training.set, kernel="linear", cost=1)  
> summary(svm.linear.C1)  
# SVM-Kernel: linear  
# cost: 1  
# Number of Support Vectors: 110 ( 40 41 7 12 5 5 )  
  
> svm.linear.C10 <- svm(Type~., data=training.set, kernel="linear", cost=10)  
> summary(svm.linear.C10)  
# SVM-Kernel: linear  
# cost: 10  
# Number of Support Vectors: 102 ( 37 39 6 12 3 5 )  
  
> svm.linear.C1000 <- svm(Type~., data=training.set, kernel="linear", cost=1000)  
> summary(svm.linear.C1000)  
# SVM-Kernel: linear  
# cost: 1000  
# Number of Support Vectors: 97 ( 34 37 6 12 3 5 )
```

Wniosek: Im większa wartość parametru kosztu C tym mniejszy margines i tym mniej wektorów nośnych.

SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Porównanie wyników dla różnych funkcji jądrowych  
  
svm.poly2 <- svm(Type~., data=training.set, kernel="polynomial", degree = 2)  
svm.poly4 <- svm(Type~., data=training.set, kernel="polynomial", degree = 4)  
svm.radial <- svm(Type~., data=training.set, kernel="radial")  
  
pred.svm.poly2 <- predict(svm.poly2, newdata=test.set)  
pred.svm.poly4 <- predict(svm.poly4, newdata=test.set)  
pred.svm.radial <- predict(svm.radial, newdata=test.set)  
  
(acc.svm.lin <- sum(diag(table(pred.svm.lin, real.labels)))/n.test)  
# 0.5694444  
(acc.svm.poly2 <- sum(diag(table(pred.svm.poly2, real.labels)))/n.test)  
# 0.4305556  
(acc.svm.poly4 <- sum(diag(table(pred.svm.poly4, real.labels)))/n.test)  
# 0.3888889  
(acc.svm.radial <- sum(diag(table(pred.svm.radial, real.labels)))/n.test)  
# 0.5833333
```

SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Optymalizacja (tuning) parametrów  
  
# zakresy dla parametrów  
> C.range <- 10^(seq(from=-3, to=3, length.out=10))  
> gamma.range <- seq(from = .01, to = 10, length.out = 20)  
  
> radial.tune <- tune(svm, train.x=training.set[,-10],  
train.y=training.set[,10], kernel="radial",  
ranges=list(cost=C.range, gamma=gamma.range))  
  
> print(radial.tune)  
# Parameter tuning of 'svm':  
# - sampling method: 10-fold cross validation  
# - best parameters:  
#   cost gamma  
#   1000 0.01  
# - best performance: 0.3166667
```

SVM w pakiecie R

Przykład: klasyfikacja odłamków szkła ($n = 214, p = 9, K = 6$)

```
#####  
# Dopasowujemy końcowy model dla optymalnych parametrów  
  
> C.best <- radial.tune$best.parameters[["cost"]]  
> gamma.best <- radial.tune$best.parameters[["gamma"]]  
  
> svm.radial.tuned <- svm(Type~., data = training.set, kernel = "radial",  
cost = C.best, gamma = gamma.best)  
  
#####  
# Prognozowanie na bazie optymalnego modelu  
  
pred.svm.radial.tuned <- predict(svm.radial.tuned, newdata=test.set)  
(acc.svm.radial <- sum(diag(table(pred.svm.radial.tuned, real.labels)))/n.test)  
# 0.7638889
```